

DEDICATED TO INNOVATION

www.bacancy.com





**Customer Satisfaction
Is Our Highest Priority**



Employee Matters



In this session

What we are learning in this session

- ▶ Entity States
- ▶ Eager Loading
- ▶ Lazy Loading
- ▶ Explicit Loading

Let's Start

Entity States

- ▶ In Entity Framework Core (EF Core), entities can exist in various states during their lifecycle as the DbContext tracks them.
- ▶ Entity States in EF Core represent an entity's state in relation to its interaction with the DbContext. EF Core uses these states to determine what operations need to be performed on the database when SaveChanges() is called.
- ▶ **Added:** The entity is being tracked by the context but does not yet exist in the database. When SaveChanges() is executed, EF Core issues an INSERT command for this entity.
- ▶ **Unchanged:** The entity is being tracked and exists in the database but has not been modified since it was retrieved. EF Core will not issue any database commands for entities in this state during SaveChanges().

Entity States

- ▶ **Modified:** The entity is being tracked and exists in the database, and some or all of its property values have been modified. When `SaveChanges()` is executed, EF Core issues a `UPDATE` command that only updates the changed properties.
- ▶ **Deleted:** The entity is being tracked and exists in the database but has been marked for deletion. When `SaveChanges()` is executed, EF Core issues a `DELETE` command for this entity.
- ▶ **Detached:** The entity is not being tracked by the context. EF Core will not perform any database operations on detached entities when `SaveChanges()` is called. This state is typical for entities that have been created but not yet added to the context, have been explicitly detached from the context, or have been deleted from the database.
- ▶ We can check the current state of my entity using **`_context.Entry(trainer).State;`**

Eager Loading

- ▶ In Eager Loading, related data (such as Branch, Address, and Courses) is loaded automatically as part of the initial query to the database. This is done using the `Include()` and `ThenInclude()` methods. The related entities are loaded in the same query when the Student entity is retrieved.
- ▶ This helps avoid the “**N+1 Query Problem**,” where multiple queries would otherwise be sent to the database to fetch each related entity one by one.
- ▶ Eager Loading is ideal when you know we need the related entities along with the main entity immediately and want to minimize the number of database queries.
- ▶ **When to Use Eager Loading with EF Core:**
 - You Know You’ll Need Related Data Immediately:
 - You Want to Minimize Round Trips to the Database:
 - When You Want to Avoid the N+1 Query Problem:

Lazy Loading

- ▶ In Lazy Loading, the related data is not retrieved initially. Instead, the related entities are loaded only when we access the navigation property for the first time. EF Core makes a separate query to the database when we try to access the related property.
- ▶ **How Do We Implement Lazy Loading in EF Core?**
 - Install the Required Package** - `Microsoft.EntityFrameworkCore.Proxies`
 - Enable Lazy Loading in the DbContext:** `optionsBuilder.UseLazyLoadingProxies();`
 - Mark Navigation Properties as virtual**
- ▶ With Lazy Loading enabled in EF Core, EF Core will automatically load the data from the database when we access a related entity for the first time. The loaded data is then stored in the context's change tracker (in memory). If we reaccess the same related entity within the same DbContext instance, the data is fetched from in-memory rather than querying the database again. This avoids redundant database calls and improves performance for subsequent access to the same entity.

Explicit Loading

- ▶ In Lazy Loading, the related data is not retrieved initially. Instead, the related entities are loaded only when we access the navigation property for the first time. EF Core makes a separate query to the database when we try to access the related property.
- ▶ **How Do We Implement Lazy Loading in EF Core?**
 - Install the Required Package** - `Microsoft.EntityFrameworkCore.Proxies`
 - Enable Lazy Loading in the DbContext:** `optionsBuilder.UseLazyLoadingProxies();`
 - Mark Navigation Properties as virtual**
- ▶ With Lazy Loading enabled in EF Core, EF Core will automatically load the data from the database when we access a related entity for the first time. The loaded data is then stored in the context's change tracker (in memory). If we reaccess the same related entity within the same DbContext instance, the data is fetched from in-memory rather than querying the database again. This avoids redundant database calls and improves performance for subsequent access to the same entity.

Thank you